

Agentic AI: A Comprehensive Study of Autonomous AI Agents, Their Architectures, Applications, Challenges, and Future Directions

Author: Author Name -- to be filled

Department: Department Name -- to be filled

College: College Name, SPPU-affiliated -- to be filled

University: Savitribai Phule Pune University (SPPU)

Academic Year: Academic Year -- to be filled

FRONT MATTER

Agentic AI: A Comprehensive Study of Autonomous AI Agents, Their Architectures, Applications, Challenges, and Future Directions

Author: [Author Name -- to be filled] Department: [Department Name -- to be filled]

College: [College Name, SPPU-affiliated -- to be filled] University: Savitribai Phule Pune

University (SPPU) Academic Year: [Academic Year -- to be filled]

No formatting specification is claimed to be officially mandated by SPPU; the document follows a conventional academic engineering research-paper structure and is kept easy to adapt should the college provide an official template.

[Certificate of original work and guide approval -- text to follow the institute's prescribed format.]

[Student declaration of originality -- text to follow the institute's prescribed format. The declaration covers, among other work, the pilot experiment (one run per condition) and its logs as the authors' own observations.]

The authors thank their project guide and department for supervision of this work, and the open-source communities behind Spring Boot, H2, and JUnit, whose software made the pilot experiment possible.

Autonomous AI agents -- systems that pursue goals through reasoning, tool use, and environmental feedback -- are surveyed alongside a small controlled pilot experiment comparing three agentic workflows. The review synthesizes eight verified sources covering agent taxonomies, the ReAct reasoning-and-acting paradigm, conversational multi-agent frameworks, and agent-evaluation methodology. The experiment assigns the same Student Management REST API construction task (Java 21, Spring Boot 4.1.1, H2) to three conditions -- Direct LLM generation, a ReAct-style reason->act->observe loop, and a Manager/Developer/Tester multi-agent system -- each judged by an identical fixed 12-test HTTP evaluation suite plus one context test. The observed result is uniform: all three conditions passed 12/12 evaluation tests (13/13 overall) with zero human interventions. Process records differ structurally -- 3 logged iterations for Direct LLM (including two pre-harness Boot 4 compatibility fixes), 7 ReAct cycles, and 4 multi-agent interactions over 1 coordination cycle -- with single-run Maven times of 22.045 s, 21.753 s, and 26.963 s.

With one run per condition (n=1), no statistical significance is claimed and no architecture is ranked; the pilot demonstrates a reproducible evaluation methodology and feasibility on a scoped task, and its limits motivate larger replicated studies. Supporting artifacts -- a frozen task specification, a shared Boot 4 test harness, hash-verified suite distribution, and full procedural logs -- are committed alongside the paper as reusable method records.

Agentic AI, AI Agents, ReAct, Multi-Agent Systems, Large Language Models, Software Engineering, AI Evaluation, Autonomous Agents

1. Introduction (1.1 Background ... 1.9 Organization of the Paper) -- 2. Literature Review (2.1-2.9) -- 3. Agentic AI Fundamentals (3.1-3.10) -- 4. Agentic AI Architectures (4.1-4.6) -- 5. Applications, Challenges and Security (5.1-5.14) -- 6. Experimental Methodology (6.1-6.14) -- 7. Experimental Results (7.1-7.9) -- 8. Discussion (8.1-8.8) -- 9. Future Directions (9.1-9.8) -- 10. Conclusion (10.1-10.3) -- References -- Appendices A-D. Section titles are as headed in the body.

Figure 1 -- Evaluation test-pass comparison (single-run values; n=1 per condition; no error bars). Figure 2 -- Full-suite Maven execution-time comparison (single-run values; n=1; not a benchmark). Figure 3 -- Process-level measurements (iterations, cycles, interactions; distinct units -- do not aggregate). Source data: results/figures-data.md.

Table 1 -- Experimental environment (Java 21, Spring Boot 4.1.1, Maven, H2). Table 2 -- Evaluation test categories (12 tests + 1 context test). Table 3 -- Consolidated experimental results (n=1 per condition). Table 4 -- Process-level measurements (distinct units).

LLM -- Large Language Model. ReAct -- Reasoning and Acting. API -- Application Programming Interface. REST -- Representational State Transfer. JPA -- Jakarta Persistence API. MVC -- Model-View-Controller. CRUD -- Create, Read, Update, Delete. H2 -- Java-embedded relational database used here in-memory. RQ -- Research Question.

CHAPTER 1 -- INTRODUCTION

Recent progress in large language models has shifted attention from single-turn text generation toward systems that act autonomously to achieve goals. Such autonomous agents perceive their environment, reason over observations, invoke tools, and adjust their behavior on the basis of feedback [3, 4]. Unlike a conventional classifier or a conversational assistant that responds to one prompt at a time, an agent is expected to carry a multi-step task through

to completion, for example by reading files, editing code, running commands, and interpreting the results. This paper studies that class of systems -- referred to here as agentic AI, following the taxonomy of Sapkota et al. [3] and the survey of Abou Ali et al. [4] -- through two complementary contributions: a synthesis of the established literature, and a small controlled pilot experiment in which three different agentic approaches solve the same software-engineering task. The pilot is presented strictly as an illustration of a reproducible comparison method, not as evidence for general properties of agentic systems.

Traditional artificial intelligence systems were typically built for a fixed task with a fixed input-output contract: rule-based programs encoded expert knowledge explicitly, while classical machine-learning models learned a mapping from training data to predictions. Both operated within narrowly defined boundaries and could not easily decompose novel problems, use external tools, or recover from their own mistakes. The arrival of large language models changed the interface -- a single model could follow open-ended instructions -- but single-turn inference on its own still lacked persistence, environment interaction, and error recovery. Tool-using agents closed part of this gap by giving models access to file systems, compilers, test runners, and web interfaces, and agentic AI, as surveyed by Abou Ali et al. [4] and taxonomized by Sapkota et al. [3], denotes the resulting systems that combine goal-directed planning, reasoning, memory, tool use, and observation into an autonomous loop. Throughout this paper, statements about this evolution drawn from the literature are cited as such, while statements about our own pilot are labeled as our observations.

Architectural proposals for agents -- single-pass generation, reasoning-and-acting loops, and multi-agent collaboration -- are frequently compared informally or on different tasks with different success criteria, which makes it difficult to attribute any observed difference to the architecture itself. What is needed, and what this paper pilots at small scale, is a head-to-head comparison in which the task, the starting code, the environment, and the pass/fail criteria are held constant while only the agentic approach varies. Concretely, this paper compares three approaches -- Direct LLM generation, a ReAct-style reasoning/action loop, and a multi-agent Manager/Developer/Tester system -- on one identical task: a Student Management REST API defined by a frozen specification and judged by a fixed 12-test HTTP evaluation suite (plus one default context test).

The motivation is both pedagogical and practical. In an engineering-college setting, students increasingly complete software tasks with LLM assistance, yet project evaluation still needs a reproducible basis for judging what was built and how. A frozen task specification paired

with an automated, hash-verified test suite provides exactly such a basis, and the same apparatus doubles as a research instrument for comparing agentic strategies fairly. No claim is made here about the scale of industrial adoption of these techniques.

The experiment is designed to answer only the following questions, and no additional questions beyond its reach are posed:

RQ1: How do Direct LLM, ReAct, and Multi-Agent approaches perform on the same multi-step software-engineering task under a common evaluation suite?

RQ2: What process-level differences can be observed between the three approaches?

RQ3: What limitations arise when evaluating agent architectures using a small controlled software-engineering task?

The objectives of this paper, each directly achievable within its scope, are: (i) review Agentic AI concepts and architectures from the verified literature; (ii) examine the ReAct and multi-agent approaches in particular; (iii) review existing agent-evaluation approaches; (iv) design a controlled pilot experiment with a frozen task and fixed suite; (v) compare the three approaches on the same Student Management REST API task; (vi) evaluate functional correctness together with observable process metrics (iterations, cycles, coordination records, and recorded run times); and (vii) identify limitations and future research directions warranted by the pilot.

The study focuses on software-engineering agents and a single representative task: a Java 21 / Spring Boot 4.1.1 Student Management REST API backed by an in-memory H2 database, implemented under a frozen specification and verified by a fixed HTTP-level test suite. The survey component covers the eight verified references in [paper/literature-foundation.md](#). Studies involving other task domains, other technology stacks, other models, or cost and token metering are outside the scope of this paper.

The pilot has the following explicit limitations, documented here so that no later chapter can overstate the findings: it covers one task; it uses one model configuration as provided by the environment; it comprises one experimental run per condition ($n=1$); the sample size therefore supports no statistical significance testing; cost and token measurement is absent because no reliable counters were exposed; and generalizability beyond this task, stack, and machine is not established. The task itself exhibited a ceiling effect (all conditions passed), and the run order introduced a sequencing confound (Condition A absorbed the Boot 4 test-harness fixes that Conditions B and C inherited). These points are examined further in Chapter 8.

Chapter 2 reviews the literature; Chapter 3 systematizes agentic AI fundamentals; Chapter 4 details the architectures; Chapter 5 surveys applications, challenges, and security; Chapter 6 documents the experimental methodology; Chapter 7 reports the observed results; Chapter 8 discusses their interpretation; Chapter 9 outlines future directions; and Chapter 10 concludes. The References list the eight verified sources, and Appendices A-D carry the experimental specification, the fixed evaluation suite, the experimental logs, and the raw results respectively.

CHAPTER 2 -- LITERATURE REVIEW

Traditional AI systems -- expert systems with hand-encoded rules and machine-learning models trained for a single predictive mapping -- performed reliably within their specified envelope but could not readily decompose unfamiliar problems, operate software tools, or revise their own plans after failure. Large language models broadened the interface by conditioning open-ended generation on natural-language instructions, yet a single forward pass still lacks task persistence and environmental grounding. The surveyed literature characterizes the subsequent step as the coupling of such models with perception, memory, and tool interfaces so that behavior extends across multiple steps [3, 4]. This section reports that characterization as established survey framing; no benchmark figures are asserted here.

Following the taxonomy of Sapkota et al. [3], an AI agent is understood here as a single-actor system that perceives its context, reasons toward a goal, and acts through available tools -- for example, reading and writing files, executing shell commands, or querying an application under test. The agent receives observations (command output, test results, error messages) and conditions its next action on them. This definition is the working meaning of "agent" throughout the paper and the unit of analysis for Conditions A and B.

Agentic AI, in the same taxonomy, denotes systems whose autonomy spans extended multi-step pursuits rather than isolated tool calls: they maintain goals, plan and replan, coordinate sub-tasks, and integrate feedback over a full lifecycle [3, 4]. Abou Ali et al. [4] survey how such systems are architected, where they are applied, and which research directions remain open. Our usage follows this survey-level meaning, and our Condition C (a Manager/Developer/Tester collaboration) is presented as one concrete instance of it -- an experimental observation, not a redefinition.

The surveyed architectures span two broad families [4]. In the single-agent family, one model carries the whole loop of reasoning, acting, and observation. In the multi-agent family, specialized agents with distinct roles collaborate through structured conversation, sharing plans, intermediate artifacts, and verification reports [2]. The ReAct paradigm of Yao et al. [1] is the canonical example of the first family in this paper, and the conversational multi-agent framework of Wu et al. [2] is the canonical example of the second; Abou Ali et al. [4] provide the survey backbone relating them.

Yao et al. [1] introduce ReAct as the synergistic interleaving of verbal reasoning traces with actions directed at an external environment, whose observations are then fed back into subsequent reasoning. The loop -- reason, act, observe, repeat -- grounds the model's problem solving in concrete tool outcomes rather than unaided generation, and it is directly applicable to software tasks where compilation and test output serve as observations. This is the sole verified primary source for the ReAct paradigm used in this paper, and it is the theoretical basis of our Condition B, which logged seven explicit reason->action->observation cycles.

Wu et al. [2] propose enabling LLM applications through multi-agent conversation: agents with distinct roles and capabilities collaborate via dialogue, divide labor, critique intermediate results, and incorporate human input and tool use. Role separation makes each transition -- delegation, handoff, and verification -- an inspectable artifact. This is the sole verified primary source for multi-agent organization used in this paper, and it is the theoretical basis of our Condition C, instantiated as Manager (planning and review), Developer (implementation), and Tester (test execution and reporting) with one logged coordination cycle.

How agents should be judged is itself surveyed in the verified literature. Yehudai et al. [5] survey evaluation dimensions and methodological considerations for LLM-based agents, while Mohammadi et al. [6] survey benchmarking practice with emphasis on comparison methodology and the replication needed before significance can be claimed. Task-grounded benchmarks provide the precedents: AgentBench evaluates language models as agents across diverse environments [7], establishing the pattern of fixed tasks with uniform scoring that our frozen 12-test suite follows at small scale. These sources directly motivate three of our controls: identical pass/fail criteria across conditions, hash-verified copying of the evaluation suite, and the refusal to perform significance testing on single runs.

The closest precedent for our pilot is test-verified software-task benchmarking. SWE-bench measures whether language models can resolve real-world GitHub issues, with resolution

judged by tests [8]; AgentBench likewise includes settings in which agents operate on code [7]. The ReAct loop supplies the mechanism by which such tasks are typically approached -- compiler and test output as observations guiding the next edit [1]. Our Student Management REST API pilot is positioned explicitly as a small instance of this paradigm: one scoped construction task, judged by an automated suite, executed identically across conditions. It is not presented as a replacement for large-scale benchmarks such as [7] and [8].

The existing literature, as verified, provides surveys and taxonomies of agentic AI [3, 4], the ReAct architectural approach [1], the AutoGen multi-agent framework [2], agent-evaluation methodology [5, 6], and software-task benchmarks [7, 8]. What it does not provide -- and what this paper pilots rather than claims to close -- is a fully logged, same-task, same-suite, same-environment comparison of a direct-generation baseline against a ReAct agent and a role-separated multi-agent system, with frozen starting points, hash-verified evaluation copying, and per-condition process records. This pilot fills exactly one cell of that larger matrix: one task, one stack, one run per condition. It leaves open replication across tasks, models, and trials, larger and more discriminative suites, and cost and safety instrumentation, all of which are recorded as future work rather than asserted results.

CHAPTER 3 -- AGENTIC AI FUNDAMENTALS

An AI agent is defined in this paper, following Sapkota et al. [3], as a single-actor system that perceives its context, reasons toward a goal, and acts through available tools. Agentic AI denotes systems whose autonomy extends across multi-step pursuits -- maintaining goals, planning and replanning, coordinating sub-tasks, and integrating feedback over a full lifecycle [3, 4].

Goal orientation distinguishes agents from single-turn generators: the agent accepts a goal (in our pilot, "implement the specification so the fixed suite passes"), decomposes it into sub-goals (model the entity, provide persistence, expose endpoints, verify), and treats each tool outcome as progress information rather than as a final answer. The decomposition itself may be implicit in a single pass or explicit in a written plan; what matters is that behavior is organized around the goal state and terminates on verifiable criteria rather than on producing text. This characterization follows the surveyed framing [3, 4]; the parenthetical illustration is our own experimental observation.

Planning is the production of an ordered course of action before or during execution,

together with delegation of its steps. In its simplest form it is a file-creation order (entity before repository before controller); in multi-agent settings it becomes an explicit artifact assigning work to roles, as in our Condition C Manager plan, which specified files, endpoint mappings, and acceptance checks before any code was written. General claims about planning architectures are drawn from the survey literature [4]. This paper makes no claim involving formal planning theory and offers none.

Reasoning here means the intermediate inference an agent performs between observations: diagnosing a compiler error, deciding which file to create next, or interpreting a test failure. The ReAct paradigm formalizes this as explicit verbal traces interleaved with actions [1], and that formalization is the only reasoning model asserted in this paper. Our logged ReAct cycles are presented as an application of it, not as independent evidence for its generality -- a distinction maintained throughout.

Survey treatments distinguish working memory (the current context: specification text, recent tool output, the plan under execution) from persistent memory (retrievable records across runs, such as logs and reports) [4]. Our pilot relied on working memory within each run and on persistent artifacts across runs (frozen commits, hash-verified copies, Surefire logs), but implemented no explicit memory mechanism such as a vector store or episodic recall. Specific memory architectures are outside the scope of this paper and are not discussed.

Tools are the agent-environment interface: file readers and writers, the Maven build, the test runner, and hash verification in our pilot. Through tools, abstract decisions ("the controller is missing") become checkable operations ("create the file, compile, run the suite") [1, 2]. The set of tools bounds what the agent can possibly achieve, which is why our experiment fixes the tool set identically across conditions and records every tool action in its logs.

Observation is the environment's reply to an action -- compiler diagnostics, HTTP status codes, assertion outcomes -- and feedback is the use of that reply to steer the next step [1]. Test logs are the canonical feedback channel in software tasks: a validation warning resolving to an expected 4xx response confirms correct behavior, while a genuine failure would trigger diagnosis and repair. Our pilot's design provides this channel by construction; its exercise is documented in the condition logs.

The actions available in our setting were reading files, creating and editing source files, running Maven commands, copying the evaluation suite with hash checks, and inspecting results -- the full repertoire is enumerated in the condition reports as our own observation.

No broader taxonomy of agent action spaces is asserted.

Human-in-the-loop arrangements let people supervise, approve, or interrupt autonomous runs, and conversational multi-agent frameworks explicitly support such oversight [2]. Our pilot was configured for full autonomy and recorded zero interventions in every condition; that figure describes this experiment only. No quantitative claim about supervision effectiveness or typical intervention rates is made.

Abstracting across conditions, the lifecycle instantiated in this work runs: specify (frozen task) -> start from a frozen commit -> plan (explicit in Conditions B and C, implicit in A) -> implement -> copy the suite with hash verification -> test -> verify the diff -> report -> commit. This sequence is presented as our method, documented in the procedural logs, not as a universal lifecycle claimed from the literature.

CHAPTER 4 -- AGENTIC AI ARCHITECTURES

In the single-agent architecture, one model carries the entire loop: it reads the task, produces the implementation, and submits it for verification, without an explicit intermediate reasoning record or role separation. Our Condition A (Direct LLM baseline) instantiates this pattern -- a single production-quality pass over entity, repository, and controller, followed by the standard test procedure [4]. Its virtue is simplicity and minimal coordination overhead; its weakness is opacity, since the reasoning behind each decision is not separately recorded.

A tool-using agent extends the single model with grounded operations on its environment. In our setting the tools were the file system (reads, writes, hash comparisons), the Maven build (`mvn -B test-compile`, `mvn -B test`), and the test reports themselves, which convert abstract intentions into checkable outcomes [1]. Tool use is what separates an agent from a pure generator: every consequential claim ("the endpoint returns 201") can be verified by executing the suite rather than by inspecting prose.

The ReAct architecture structures behavior as repeating Reason -> Act -> Observe cycles [1]. Our Condition B logged seven such cycles: inspecting the specification and harness, creating the entity, creating the repository and controller, compiling, hash-copying the evaluation suite, running the full suite, and verifying the result. Each cycle's reasoning determined the next tool action, and each observation decided the cycle after. No claim is made here about ReAct's efficacy beyond the logged trace; the architecture is described, not

ranked.

The multi-agent architecture distributes the same work across specialized roles coordinated through explicit handoffs [2]. Our Condition C used exactly three roles: the Manager analyzed requirements and issued a written plan with acceptance criteria; the Developer created the three implementation files; the Tester inspected the result, copied the suite with hash verification, executed it, and reported back. Four interactions over one coordination cycle completed the task, and the designed recovery path (Tester->Manager->Developer on failure) remained unexercised because the first run passed. Role separation makes each transition auditable at the price of coordination overhead.

Hybrid designs -- for example, a ReAct loop executed inside each role of a multi-agent team, or a single agent that spawns subagents for subtasks -- were not implemented or tested in this work. No statement about their performance, prevalence, or best practices is made here. Hybrids are noted only as a natural direction for follow-up experiments.

The table below compares the architectures structurally. It records mechanisms and characteristic trade-offs at survey level; it does not rank the architectures, because the pilot's functional outcomes were identical across conditions (13/13 each) and any ranking would be unsupported.

Architecture	Main mechanism	Strengths	Limitations	Typical use
Single-agent	One model performs the whole task in a	Simple; no coordination overhead;	Reasoning is implicit and unaudited; no	Small, well-specified construction tasks
Tool-using	Single agent grounded through environment tools (files, build,	Claims run become verifiable by execution; less	Builds by its tool set; tool failures stall the run	Code and configuration tasks with fast feedback
ReAct	Explicit Reason -> Act -> Observe cycles [1]	Reduced risk from observation error each step	Extra logging and iteration overhead; trace quality varies	Multi-step tasks needing diagnosis from tool output
Multi-agent	Role-specialized agents with structured	Separation of concerns; checkable	Coordination overhead; recovery	Larger tasks divisible into plan/build/verify
Hybrid	Handoffs loops nested inside multi-agent roles (not tested)	Notional designated recovery path	Notional designated recovery path	Future work (not tested)

CHAPTER 5 -- APPLICATIONS, CHALLENGES AND SECURITY

Software construction is the most directly supported application in this paper, grounded both in benchmark literature and in our own pilot. Test-verified benchmarks such as SWE-bench

[8] and multi-environment evaluation such as AgentBench [7] establish the pattern our experiment follows at small scale: a scoped code task judged by an automated suite, with the ReAct loop supplying the edit-observe mechanism [1]. Our pilot instantiates it concretely -- a CRUD REST API built and verified through hash-checked test runs. No claim is made that success on this task predicts success on larger codebases.

Healthcare applications of agentic AI (triage assistance, documentation, decision support) are mentioned at survey level in the reviewed literature [4]. This paper states only that opportunity exists as a surveyed direction. No statistics are stated.

The same restriction applies to finance (analysis assistance, reporting automation, compliance support): surveyed as an application direction [4], with no statistics stated.

Education is likewise a surveyed application direction [4], and it is additionally our own institutional context: an SPPU-style project report in which agent-built software is evaluated reproducibly. That framing is our own; no claim about pedagogical effectiveness is made.

Defensive and offensive security uses of agents are surveyed directions [4]. No technical specifics are stated here.

Embodied and robotic applications are surveyed directions [4]. No specifics are stated.

Hallucination -- confident but false content -- is discussed in the surveyed literature as a framing-level challenge for autonomous systems [3, 4]. Tool grounding of the kind used in our pilot (every functional claim checked by test execution) is one surveyed mitigation direction. No incident rates, severity distributions, or mitigation benchmarks are stated.

In multi-step runs, an early mistake can compound through later steps. This risk is discussed conceptually; no general rates or formal models are stated. Our pilot is consistent with the concept without evidencing it either way: the runs that could have exhibited propagation instead passed on their first full execution, so no propagated failure was observed.

Reliability concerns repeatability and variance across runs, the central theme of the evaluation surveys [5, 6]. Our design, with one run per condition, cannot measure reliability by construction -- a limitation stated here rather than a finding. No general reliability figure for agentic systems is stated.

Prompt injection and related manipulation of agent inputs are framing-level security concerns in the surveyed literature [3, 4]. No concrete attack data, defense evaluations, or prevalence figures are stated. It is recorded as an observation that our task involved no adversarial inputs, so this pilot contributes no evidence on the point.

Agents wielding broad tool privileges risk unintended side effects. This is stated as a conceptual risk, not a measured one, and no prevalence or impact data are stated. As an observation, our agents operated with a scoped tool set (repository files, Maven, test execution) and produced no out-of-scope modifications.

No token, time-budget, or monetary-cost data was collected in any condition, as documented in all three condition reports; any cost comparison between architectures is therefore disallowed in this paper, and no external cost benchmarks are stated.

Governance -- policies, audit trails, and accountability for autonomous runs -- is a survey-level concern [3, 4]. No specific regulatory requirements or compliance claims are stated. Our own audit practice (frozen commits, full logs, hash-verified suites) is offered as method illustration, not as a governance standard.

In sum: the verified literature supports discussing hallucination, error propagation, reliability, prompt manipulation, permissions, cost, and governance as recognized challenge areas [3, 4], with evaluation rigor as the cross-cutting control [5, 6]. Quantitative safety or security claims -- rates, guarantees, or certified mitigations -- are not asserted anywhere in this paper (see the additional-source notes in paper/reference-map.md). Future possibilities in this chapter are possibilities only, not established facts.

CHAPTER 6 -- EXPERIMENTAL METHODOLOGY

Within this pilot experiment, the research question is: how do Direct LLM, ReAct, and Multi-Agent approaches perform on the same multi-step software-engineering task under a common evaluation suite? Two subordinate questions follow: what process-level differences can be observed between the three approaches, and what limitations arise when evaluating agent architectures with a small controlled task? No question beyond the reach of a single-run pilot is posed.

The objective is to produce comparable raw observations, not rankings: the same task, the same suite, and full procedural logs for each of the three conditions. Success for the methodology is defined as identical starting conditions, identical pass/fail criteria, and complete records -- regardless of which approach, if any, appears more economical in its trace.

The design freezes history before varying the architecture. A baseline commit (60365d2,

EXPERIMENT-BASELINE-FROZEN) captured the Maven project with its H2 configuration, frozen specification, and unfixed evaluation suite. A harness commit (7c88e75, COMMON-EVALUATION-HARNESS-BOOT4) added only the Boot 4 test-compatibility changes. Three branches diverged from controlled points: experiment/direct-llm (completed as de03148) from the baseline, and experiment/react (completed as 7298ade) and experiment/multi-agent (completed as a030a19) from the harness. The experimental variable is therefore the agentic workflow and architecture; the task and the evaluation environment are controlled.

Held constant across conditions: the Student Management task and its frozen requirements; Java 21 and Spring Boot 4.1.1; the Maven environment and H2 configuration; the fixed 12-test evaluation suite plus the default context test; the evaluation criteria (exact status codes, 4xx-range validation assertions, no prescribed error-body schema); separate clean experiment branches; and zero human code intervention. The fixed suite was copied into src/test by hash-verified file copy in each condition, so its contents are identical by construction rather than by assumption.

All conditions ran in the environment recorded in Table 1. No condition received a different JDK, framework, database, or build tool.

Table 1 -- Experimental Environment (recorded values only):

Component	Value
Java version	21 (java.version=21; runtime 21.0.9 observed)
Spring Boot version	4.1.1 (spring-boot-starter-parent)
Build tool	Maven 3.9.12 (mvn -B test)
Database	H2 in-memory, jdbc:h2:mem:studentdb, user sa, ddl-auto=create-drop
Test client	TestRestTemplate via spring-boot-resttestclient + spring-boot-restclient (test scope), @AutoConfigureTestRestTemplate
Machine	Single machine, Windows 11 (one run per condition)

The task (see Appendix A) defines a Student entity -- auto-generated Long id, String name required and not blank, String email required in valid format, Integer age required between 18 and 100 inclusive -- and five endpoint groups: POST /students (valid -> 201 with generated ID; invalid data rejected), GET /students (200 with a JSON array; empty database -> empty array), GET /students/{id} (200 or 404), PUT /students/{id} (200, or 404, or validation error), and DELETE /students/{id} (204 or 404). The specification deliberately

prescribes no package structure, class names, service layout, exception-handling implementation, or error-body format, so each approach solves the same requirements independently.

The suite (StudentApiEvaluationTest, 12 tests) exercises the API exclusively over HTTP with Map payloads, so it depends on no implementation class names. Each test creates its own fixture data with a random e-mail address, making the tests independent of execution order, and validation failures assert the 4xx range rather than an exact code or body schema, since error formatting was left open. Table 2 lists the categories; the methodology precedent for fixed suites and replication discipline follows Yehudai et al. [5], Mohammadi et al. [6], Liu et al. [7], and Jimenez et al. [8].

Table 2 -- Evaluation Test Categories (12 tests + 1 context test):

Category	Tests	Expected outcomes
POST /students	valid student; invalid e-mail; blank name; age below 18	201 with generated ID; 4xx; 4xx; 4xx
GET /students	all students; existing student; nonexistent student	200 array; 200 with correct data; 404
PUT /students/{id}	valid update; nonexistent ID; invalid data	200 with updated data; 404; 4xx
DELETE /students/{id}	existing student; nonexistent student	204 (then GET -> 404); 404
Context	default contextLoads	passes

Task -> Direct LLM -> implementation. On branch experiment/direct-llm, a single production-quality pass created the entity, repository, and controller directly, with no ReAct loop and no subagents. Three implementation/fix iterations were recorded -- one initial build plus two Boot 4 harness-compatibility fixes (removed TestRestTemplate package; missing RestTemplateBuilder bean) -- which must not be read as three business-logic correction cycles, since the harness did not yet exist when this condition ran.

Task -> ReAct agent -> Reason -> Action -> Observation -> completion. On branch experiment/react, a single agent worked through seven explicit logged cycles (inspect; entity; repository + controller; compile check; hash-verified copy; full test run; verification and reporting), grounding each step in tool output per Yao et al. [1]. One implementation iteration was recorded, with zero corrections.

Task -> Manager -> Developer -> Tester -> completion. On branch experiment/multi-agent, exactly three specialized subagents collaborated per Wu et al. [2]: the Manager analyzed requirements and issued a written plan with acceptance criteria; the Developer created the three implementation files; the Tester inspected the result, copied the suite with hash

verification, executed it, and reported PASS. Four interactions over one coordination cycle were logged; the designed Tester->Manager->Developer recovery path was never triggered. One implementation iteration was recorded, with zero corrections.

The metrics are exactly those of the consolidated table: evaluation tests passed/failed, overall tests passed/failed, implementation/fix iterations, agent cycles and interactions, full-suite Maven time, and human interventions. For tool/action counts and independent wall-clock time, the recorded value in every condition is "Not reliably measurable in the experimental environment." Unavailable values are never replaced with zero.

The procedure ran identically per condition: freeze the start point -> create the condition branch -> implement -> copy the suite with hash comparison -> run mvn -B test -> diagnose and fix the implementation only if the implementation fails (never the tests) -> verify the diff -> write the condition report -> commit. The loop of implement -> test -> observe -> correct -> retest was available in all conditions; only Condition A exercised it, and only for harness compatibility.

A rerun can start from the recorded commits, apply the harness diff (two test-scope dependencies, one import change, one annotation), copy the suite with hash comparison (35E7D2E3...D2BBCC50C in Conditions B and C), and execute mvn -B test. Frozen starting points, full command histories, Surefire logs, and per-condition reports are committed on their branches. One documentation caveat is recorded in the condition reports: a report file cannot contain its own commit hash before that commit exists, so final hashes are verified via git log rather than embedded in advance.

The design carries explicit threats, hidden from no reader: n=1 (one run per condition); a single software task of limited complexity; a single model configuration as provided; a task ceiling (all conditions passed, so correctness cannot discriminate); a sequencing confound (Condition A absorbed harness fixes); limited measurement of token and API cost; unavailable reliable tool-call counts; potential architecture-specific workflow effects (e.g., logging overhead differing by design); and limited generalizability beyond this task and configuration. Chapter 8 examines each in turn.

CHAPTER 7 -- EXPERIMENTAL RESULTS

Within this pilot experiment, the observed results indicate identical functional outcomes: every condition passed the fixed evaluation suite 12/12 and the overall suite 13/13, each run

ending in BUILD SUCCESS with 0 failures, 0 errors, and 0 skipped tests. The experiment does not establish anything beyond these recorded runs.

Table 3 -- Consolidated Experimental Results (recorded values only; n=1 per condition):

Metric	Direct LLM	ReAct	Multi-Agent
Evaluation tests passed	12	12	12
Evaluation tests failed	0	0	0
Overall tests passed	13	13	13
Overall tests failed	0	0	0
Implementation/fix iterations	3	1	1
Human interventions	0	0	0
Full Maven test time	22.045 s	21.753 s	26.963 s
Tool/action count	Not reliably measurable in the experimental environment.	Not reliably measurable in the experimental environment.	Not reliably measurable in the experimental environment.

On experiment/direct-llm (completed as de03148): 12/12 evaluation tests passed, 13/13 overall, BUILD SUCCESS in 22.045 s, approximate wall-clock interval 17:53:28-18:06:06, 0 human interventions. Three iterations were recorded: the initial implementation plus two Boot 4 compatibility fixes (the relocated TestRestTemplate package and the missing RestTemplateBuilder dependency). No business-logic correction cycle was required once the harness existed, and no test was modified, skipped, or weakened.

On experiment/react (completed as 7298ade): 12/12 evaluation tests passed, 13/13 overall, BUILD SUCCESS in 21.753 s (with a prior test-compile check in 6.456 s), 0 human interventions. Seven ReAct cycles were logged and one implementation iteration was recorded, with zero corrections: compilation succeeded on the first attempt and the first full test run passed, showing only the expected validation warnings for the negative cases (blank name, malformed e-mail, underage input) resolving to 4xx responses.

On experiment/multi-agent (completed as a030a19): 12/12 evaluation tests passed, 13/13 overall, BUILD SUCCESS in 26.963 s (Surefire: 15.19 s evaluation + 1.317 s context), 0 human interventions. Four agent interactions over one coordination cycle were logged (Manager 2, Developer 1, Tester 1), one implementation iteration was recorded, and zero correction cycles occurred: the Tester's first run reported PASS, so the Manager issued no correction request and the Developer performed no fix.

Table 3 above reproduces the consolidated record verbatim: no averages across trials (there

was one trial per condition), no derived percentages, no confidence intervals, and no ranking. Commit references are 60365d2 (baseline), 7c88e75 (common harness), de03148 (A), 7298ade (B), and a030a19 (C). The table is the complete quantitative content of this chapter; everything that follows interprets process structure, not additional numbers.

```
Evaluation passed (max 12): Direct LLM [12] ##### ReAct [12] ##### Multi-Agent [12] #####
Evaluation failed:          Direct LLM [0] ReAct [0] Multi-Agent [0]
```

Figure 1 -- Evaluation test-pass comparison (single-run values; n=1 per condition; no error bars; source: results/figures-data.md):

```
Direct LLM [22.045 s] #####
ReAct      [21.753 s] #####
Multi-Agent[26.963 s] #####
```

Figure 2 -- Full-suite Maven execution-time comparison (single-run values; n=1; one machine; not a benchmark):

```
Iterations:    Direct LLM [3] ReAct [1] Multi-Agent [1]
Agent cycles:  ReAct [7] (not applicable elsewhere)
Interactions:  Multi-Agent [4 over 1 cycle] (not applicable elsewhere)
Interventions: 0 in all conditions
```

Figure 3 -- Process-level measurements (distinct units -- do not aggregate; Direct LLM iterations include 2 pre-harness fixes):

The three conditions differ in logged structure, not in functional outcome. Direct LLM proceeded as a single pass with no cycles or roles. ReAct made its work explicit in 7 reason->action->observation cycles. Multi-Agent distributed the same work across 4 role interactions in 1 coordination cycle with an unexercised recovery path. Table 4 records these side by side with the warning that Direct LLM's iteration count of 3 reflects pre-harness history (two infrastructure fixes), not three rounds of functional repair, and must not be compared naively with the single post-harness iterations of Conditions B and C.

Table 4 -- Process-Level Measurements (recorded values only; units differ -- do not aggregate):

Measure	Direct LLM	ReAct	Multi-Agent
Implementation/fix iterations	3 (1 build + 2 harness fixes)	1 (0 corrections)	1 (0 corrections)
Agent cycles	Not applicable	7 ReAct cycles	Not applicable
Agent interactions (Manager/Developer/Tester)	Not applicable	Not applicable	2 / 1 / 1 (total 4)
Coordination cycles	Not applicable	Not applicable	1
Correction cycles	0 business-logic corrections	0	0
Human interventions	0	0	0

The observed single-run Maven times are 22.045 s (Direct LLM), 21.753 s (ReAct), and 26.963 s (Multi-Agent), each on the same single machine. The experiment does not establish

that any architecture is faster: with one sample per condition, any difference is indistinguishable from run-to-run noise in JVM startup, framework initialization, and the embedded database. No error bars are shown, no average is computed, and no timing rank is asserted.

The observed result is uniform: 0 interventions in every condition -- no human wrote, selected, or repaired code. Within this pilot experiment, this indicates that a scoped, well-specified CRUD task with fast automated feedback could be completed autonomously in this environment. It does not establish supervision requirements for ambiguous, large, or safety-critical tasks, where intervention behavior would need separate measurement.

This chapter's numbers are bounded by what was reliably measurable: pass/fail counts, iteration and cycle logs, Maven durations, and intervention counts. Tool and action counts were not reliably measurable in the experimental environment; independent wall-clock measurement was unavailable; token and API-cost data were not collected. These gaps are restated here so that Chapter 8 cannot inadvertently reason beyond them.

CHAPTER 8 -- DISCUSSION

The results admit two layers that must not be confused. The observational layer is fixed: all three conditions passed 12/12 evaluation tests and 13/13 overall with zero interventions, while logging different process structures (a single pass; 7 ReAct cycles; 4 multi-agent interactions over 1 cycle) and different single-run Maven times (22.045 s, 21.753 s, 26.963 s), as recorded in results/analysis.md §1. The interpretive layer -- what these facts mean -- is where restraint is required: with one run per condition, every process difference is a documented occurrence, not a measured tendency, and every timing difference is a single sample, not a benchmark. This chapter therefore treats each interpretation as provisional and ties it explicitly to the observation it rests on.

The fixed 12-test suite did not differentiate the three conditions: Direct LLM 12/12, ReAct 12/12, Multi-Agent 12/12. Within this pilot experiment, the observed result is a ceiling effect -- the task, at this level of scope and with this suite, was solvable by every approach attempted. This limits what correctness can say here: it demonstrates feasibility on a scoped CRUD task, not relative capability. Discriminative benchmarks in the literature, such as AgentBench [7] and SWE-bench [8], exist precisely because harder, broader suites separate approaches that easy suites do not; our suite was designed for controlled comparability, not

for discrimination, and its ceiling is a property of that design choice.

The process records do differ, and the differences are worth describing as long as they are not mistaken for proof of superiority. Direct LLM shows 3 recorded iterations, but two were Spring Boot 4 test-harness compatibility fixes (the relocated TestRestTemplate package and the missing RestTemplateBuilder dependency), not business-logic corrections -- a sequencing artifact of running before the common harness existed. ReAct shows 7 explicit cycles with 1 implementation and 0 corrections; Multi-Agent shows 4 role interactions over 1 coordination cycle, likewise with 0 corrections. The honest reading is structural, not evaluative: one approach left no intermediate trace, one externalized its reasoning at each step, and one distributed the work across auditable handoffs. None of these facts establishes that any workflow is generally more efficient, because effort was not measured on a common scale and the sample is singular.

The seven logged ReAct cycles (inspect; entity; repository plus controller; compile check; hash-verified copy; full test; verification) instantiate the Reason -> Act -> Observe loop of Yao et al. [1] in a concrete setting: each cycle's reasoning selected the next tool action, and each observation (compiler output, hash match, Surefire report) determined what followed. The value demonstrated here is traceability -- an independent reader can replay why each step happened -- rather than efficacy, since the final outcome matched the unlogged baseline. Whether the trace would pay for itself on a task requiring genuine diagnosis (where observations contradict expectations) is untested in this pilot and is recorded as an open question, not a finding.

The Manager/Developer/Tester workflow of Wu et al. [2], instantiated as plan -> implement -> test -> review, executed cleanly: the plan specified files and acceptance criteria, the Developer produced exactly those files, and the Tester returned hash equality plus a PASS report with exact Surefire numbers, after which the Manager approved completion. The checkable handoffs are the demonstrated strength. The designed recovery machinery -- Tester failure report -> Manager correction request -> Developer fix -> retest -- idled throughout (0 corrections), so this pilot provides no evidence about coordination quality under failure. A multi-agent run that never needs its repair loop cannot speak to how well that loop repairs.

The pilot's components map directly onto verified precedents without extending them. Its loop follows Yao et al. [1]; its role separation follows Wu et al. [2]; its fixed-suite discipline and its refusal to claim significance from single runs follow the evaluation methodology surveyed by Yehudai et al. [5] and Mohammadi et al. [6]; and its test-verified code-task

format follows the benchmarking pattern of Liu et al. [7] and Jimenez et al. [8]. No resemblance beyond this mapping is asserted.

Three implications are licensed, and only these. First, the method works: a frozen specification, a shared Boot 4 harness, hash-verified suite copying, and full procedural logs produced directly comparable raw observations across three architectures. Second, routine scoped construction -- a validated CRUD API with fast automated feedback -- proved autonomously completable in this environment under all three workflows. Third, nothing here licenses deployment, pedagogical, or procurement generalizations: no cost data, no adversarial testing, no multi-task evidence, and no replication exist in this record.

The threats are stated without softening, following results/analysis.md §10. The sample is $n=1$ per condition, admitting no variance estimate. The task is single, of limited complexity, and set a ceiling that removes discriminative power. The model configuration is singular as provided. The run order confounds the iteration comparison, as documented. Token and API-cost measurement is absent, reliable tool-call counts were unavailable, and architecture-specific workflow effects (such as logging overhead) cannot be separated from task effort. Generalizability beyond this stack, machine, and task is therefore not established, and researcher degrees of freedom in harness repair -- though functionally neutral and shared by later conditions -- are disclosed rather than minimized.

CHAPTER 9 -- FUTURE DIRECTIONS

Future work should employ larger, more discriminative suites -- additional endpoints, authentication, pagination, concurrent access, and adversarial inputs -- that can separate approaches where a 12-test suite cannot. AgentBench [7] and SWE-bench [8] are the precedents for such benchmarks; no difficulty statistics or expected score gaps are asserted here, since none were measured.

A single CRUD task cannot represent software engineering. Follow-up studies should span multiple domains (for example, stateful workflows, integration with external services, and defect repair in existing codebases). No coverage claim beyond the tested task is asserted as established.

Repeating the same frozen protocol across models would test whether the observed patterns are model-specific. No model-comparison statistics are stated; the direction is proposed, not evidenced.

The most urgent extension is replication: pre-registered repeated trials of each condition with reported variance and confidence intervals, following the benchmarking discipline surveyed by Yehudai et al. [5] and Mohammadi et al. [6]. No such intervals are computed in this paper, and none are implied.

Reliable metering of tokens, API calls, wall-clock budgets, and monetary cost would allow genuine efficiency comparison. None of these was collected here -- a documented gap, not data -- so cost-aware evaluation is proposed as instrumentation to build, not as a result to report.

Adversarial prompt handling, permission scoping, and side-effect auditing should be evaluated deliberately, since the present task contained no adversarial inputs. No quantitative safety targets or guarantees are stated; they are proposed as evaluation to design, not forecasts to state.

Intervention-rate studies -- how often, at which points, and to what effect humans must step in -- would extend the present zero-intervention observation into a real measure, building on the human-in-the-loop support described by Wu et al. [2]. No effectiveness figures for such collaboration are asserted.

The methodological contribution proposed for reuse is the evaluation discipline itself: frozen specifications, shared Boot-version harnesses, hash-verified suite copying, full procedural logs, and raw-data-first reporting, consistent with the standards surveyed in [5] and [6]. Each item above is a proposed research direction, not a prediction presented as fact.

CHAPTER 10 -- CONCLUSION

This paper surveyed agentic AI foundations, architectures, and evaluation from eight verified sources, and piloted a controlled comparison in which Direct LLM, ReAct, and Multi-Agent approaches implemented the same Student Management REST API under an identical fixed suite. The headline numbers are: 12/12 evaluation tests and 13/13 overall tests passed in every condition; implementation iterations of 3, 1, and 1 (with the first figure including pre-harness infrastructure fixes); process traces of a single pass, 7 ReAct cycles, and 4 multi-agent interactions over 1 coordination cycle; single-run Maven times of 22.045 s, 21.753 s, and 26.963 s; and zero human interventions -- all with n=1 per condition and no significance testing.

The contributions are framed conservatively, matching only what was done: (1) a structured review of Agentic AI architectures and applications from the verified literature; (2) a controlled pilot methodology for comparing agent workflows with frozen starts and identical criteria; (3) a reproducible Student Management REST API evaluation task with its Boot 4 harness; (4) a fixed architecture-independent 12-test suite with hash-verified distribution; (5) empirical observations from the three implemented conditions with full logs; and (6) an explicit record of limitations and evaluation requirements for future studies. No new algorithm, framework, benchmark, or breakthrough is claimed.

Answering the research questions directly: RQ1 -- the three approaches all completed the evaluated task successfully, and functional correctness did not differentiate them in this pilot; RQ2 -- process-level differences were observed (single pass versus 7 logged cycles versus 4 interactions over 1 cycle, with the stated sequencing caveat on iteration counts); RQ3 -- a small single-task, single-run design can demonstrate feasibility and method but is insufficient to establish general superiority, efficiency, or reliability. Larger and repeated experiments -- multiple tasks, multiple trials and models, discriminative suites, cost metering, safety evaluation, and human-in-the-loop measurement -- are required before any general claim about agentic architectures can be responsibly made.

REFERENCES

Only the eight externally verified references are listed, with bibliographic fields exactly as recorded in paper/literature-foundation.md. No field has been invented; where the verified record supplies no venue, none is shown. Any future source must first be externally verified and recorded in the foundation file.

- [1] Yao, Shunyu, et al. "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023. arXiv:2210.03629.
- [2] Wu, Qingyun, et al. "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation." arXiv:2308.08155.
- [3] Sapkota, Ranjan, Konstantinos I. Roumeliotis, and Manoj Karkee. "AI Agents vs. Agentic AI: A Conceptual Taxonomy, Applications and Challenges." Information Fusion, Vol. 126, Part B, Article 103599, 2026. DOI: 10.1016/j.inffus.2025.103599.
- [4] Abou Ali, Mohamad, Fadi Dornaika, and Jinan Charafeddine. "Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions." Artificial

Intelligence Review, Vol. 59, Article 11, 2026. DOI: 10.1007/s10462-025-11422-4.

- [5] Yehudai, Asaf, et al. "Survey on Evaluation of LLM-based Agents." arXiv:2503.16416, 2025.
 - [6] Mohammadi, Mahmoud, Yipeng Li, Jane Lo, and Wendy Yip. "Evaluation and Benchmarking of LLM Agents: A Survey." arXiv:2507.21504, 2025.
 - [7] Liu, Xiao, et al. "AgentBench: Evaluating LLMs as Agents." arXiv:2308.03688.
 - [8] Jimenez, Carlos E., et al. "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" ICLR 2024. arXiv:2310.06770.
-

APPENDICES

The frozen task specification is committed as experiment-specification.md: a Student entity (auto-generated Long id; name required, not blank; email required, valid format; age required, 18-100 inclusive) with five endpoint groups -- POST /students (201 + generated ID; invalid data rejected), GET /students (200 array; empty database -> empty array), GET /students/{id} (200/404), PUT /students/{id} (200/404/validation error), DELETE /students/{id} (204/404). Package structure, class names, and error-body format were deliberately unspecified. The appendix reproduces that file by reference without paraphrase.

The suite is committed as evaluation-tests/StudentApiEvaluationTest.java (Boot 4 form: org.springframework.boot.testclient.TestRestTemplate,

@AutoConfigureTestRestTemplate, test-scope spring-boot-testclient/spring-boot-testclient). Design: HTTP-level tests with Map payloads, per-test random e-mails, 4xx-range validation assertions. Distribution to each condition was by hash-verified copy (recorded prefix 35E7D2E3..., full hashes in the condition reports).

Procedure records are committed per condition: results/react/react-log.md (7 reason->action->observation cycles with commands and observations) and results/multi-agent/agent-log.md (4 interactions: Manager planning, Developer implementation, Tester execution and PASS report, Manager approval), together with the command histories and Surefire excerpts in results/direct-llm/experiment-report.md, results/react/experiment-report.md, and results/multi-agent/experiment-report.md.

Primary data: results/consolidated-results.md (the authoritative table), results/figures-data.md (chart-ready values), and the three condition reports. Version

control: baseline 60365d2, harness 7c88e75, Condition A de03148, Condition B 7298ade,
Condition C a030a19.